Predictive Evaluation Competition: Methods Tried, Hosted Results, and Final Selection

Competition Technical Report

June 2026

Abstract

This report summarizes the submission cycle for the Codabench Predictive Evaluation Competition [3]. The competition ranks submissions by hosted negative log-loss, with AUC-ROC as a secondary diagnostic metric. Across 82 parsed hosted rows (80 finished scored rows), the best rounded hosted score remained -0.62. The selected final submission is `submission_domain_live_mid600.zip`, with hosted score -0.62, approximate log-loss 0.62, and competition-reported AUC-ROC 0.72. The main empirical conclusion is that compact domain-prior submissions transferred better than high-AUC local stacks because hidden test items came from benchmark families absent from public training. Later iterations increased local confidence and ranking metrics, but the hosted log-loss reward favored conservative calibration under benchmark shift.

1 Executive Summary

This competition submission report is written around the hosted Codabench evidence and the final ZIP that would be submitted or defended in the Predictive Evaluation Competition.

The final selected method is `submission_domain_live_mid600.zip`. It is a compact formula-only domain-prior runtime with no adaptive labeling policy, no exact item-memory lookup, no public benchmark lookup, and no tree or forecast ensemble at inference time. Its hosted score was -0.62, corresponding to an approximate log-loss of 0.62. The AUC-ROC value for this selected row is 0.72, reported by the competition interface because `eval_finals.txt` stores only the hosted score.

The original target was hosted log-loss below 0.60, equivalent to a displayed negative log-loss above -0.60. That target was not reached. The result is still informative: the hosted leaderboard repeatedly favored conservative domain-prior variants clustered around -0.62, while higher-complexity AUC, forecast, tree, item-memory, and hierarchical OOD variants generally scored from -0.63 to -0.71. The degradation pattern is consistent with the competition constraint that hidden items are drawn from benchmarks not present in public training.

The submission cycle therefore settled on the lower-risk candidate rather than continuing to spend hosted attempts on increasingly confident local proxies. The best practical explanation is calibration under distribution shift: hidden log-loss punishes overconfident wrong probabilities, while public-slice AUC can look excellent when benchmark, subject, condition, or item overlap remains in the validation slice.

2 Problem Setup And Metric

The task is amortized prediction of subject-item correctness in the hosted Codabench competition. At runtime, the platform passes each hidden subject-item pair to the submitted predictor. The

visible input contains benchmark, condition, subject content, and item content. The output is a probability of binary correctness [2, 3].

The primary metric is negative log-loss:

$$\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right].$$

It is a negative mean log-likelihood, bounded above by zero, so higher leaderboard values are better. A score of -0.62 means an ordinary log-loss of about 0.62. AUC-ROC is secondary. This distinction matters because AUC measures ranking, but log-loss also penalizes probability scale and calibration.

The hosted hidden-eval setup also constrained what could transfer. The public training data came from the Measurement DB dataset [1], but hidden scoring rows were served only through the runtime. The starter kit states that hidden test items come from benchmarks not present in public training; therefore, exact public item memory and public benchmark-specific priors were structurally high risk [2].

3 Data And Hidden-Evaluation Constraints

The active hidden evaluation sampled 5000 hidden items per submission, stratified by organizer data category. The data-category field was not passed to the predictor. If a submission did not include an acquisition policy, the platform chose the same number of labels uniformly at random; if a policy was present, the platform used acquisition scores subject to the same category budget. Because the category was hidden, adaptive labeling could easily optimize a public proxy while skewing the small revealed-label set under hosted sampling.

The most important constraint was benchmark-family absence. Public rows were useful for estimating broad subject behavior and coarse item-domain structure, but they were not a reliable source of exact hidden item or benchmark identity. This made cold benchmark-family validation more relevant than row holdout or public-slice probes.

4 Methods Tried

4.1 Notation And Shared Calibration

Let s denote a subject, i an item, and $y_{s,i} \in \{0,1\}$ the hidden correctness label. Every method produced a probability $p_{s,i}$ and then clipped it to $[\epsilon, 1-\epsilon]$ to avoid infinite log-loss. Most empirical means used the same shrinkage pattern:

$$\text{shrink}(\bar{y}_g, n_g; c, \tau) = \frac{n_g \bar{y}_g + \tau c}{n_g + \tau},$$

where g is a group such as a subject, item, or domain, c is a global center, n_g is the number of public observations in the group, and τ controls how aggressively the estimate is pulled back toward the global center. This shrinkage was important because hidden benchmark families were not present in public training.

Several candidates also used a one-dimensional calibration transform:

$$p' = \sigma\left(\alpha + \beta\ \text{logit}(p)\right),$$

where σ is the logistic function. Values of $\beta < 1$ cool the prediction spread; values of $\beta > 1$ make predictions sharper. The hosted results consistently favored cooler, more conservative calibration.

4.2 Global Centers, Subject Priors, And Calibration

The first family tested whether the hidden leaderboard could be explained mostly by base rate and subject ability. The simplest model was a constant:

$$p_{s,i} = c.$$

The next variant added a subject ability term estimated from public observations for the same visible subject name:

$$\hat{p}_s = \frac{n_s \bar{y}_s + \tau_s c}{n_s + \tau_s}, \qquad p_{s,i} = \text{clip}\left((1-\lambda_s)c + \lambda_s \hat{p}_s, \epsilon, 1-\epsilon\right),$$

with $\lambda_s = n_s/(n_s + \tau_s)$. In words, subjects with many public observations received a stronger subject-specific estimate; low-support or ambiguous subjects stayed close to the global rate. Alias matching was used to connect visible subject descriptions to public subject records, but the method intentionally avoided relying on hidden item identity.

These baseline and calibration variants produced hosted scores around -0.63 to -0.66. Their main value was diagnostic: the hidden task rewarded calibrated base rates more than highly specialized public-row fits, but subject priors alone did not add enough item information to beat the -0.62 plateau.

4.3 Benchmark, Condition, And Light Engineered Priors

Early engineered variants added public benchmark and condition means when a public group was available:

$$p_{s,i} = c + a_s + b_{\text{bench}(i)} + q_{\text{cond}(i)},$$

where a_s is a subject adjustment, b is a benchmark adjustment, and q is a condition adjustment. These terms were always shrunk toward zero. This family was useful on public validation slices, but it had an obvious transfer risk: the competition hidden items are from benchmark families absent from public training. When a hidden benchmark has no public rows, exact benchmark and condition priors either disappear or fall back to broad averages. That made this family more of a calibration baseline than a final strategy.

4.4 Semantic And Text Difficulty Models

The semantic/text family tried to infer item difficulty from visible item content and subject descriptors. A generic form was:

$$z_{s,i} = \beta_0 + \beta_s^\top \phi_s(x_s) + \beta_i^\top \phi_i(x_i) + \beta_{si}^\top \phi_{si}(x_s, x_i), \qquad p_{s,i}^{\text{text}} = \sigma(z_{s,i}),$$

where x_s is subject text, x_i is item text, and ϕ represents text-derived features such as domain keywords, task type cues, difficulty indicators, subject family indicators, and semantic similarity features. Some variants used amortized difficulty scores from text models; others used simpler keyword and domain features.

The text estimate was usually blended with a conservative prior:

$$p_{s,i} = (1-\eta) p_{s,i}^{\text{prior}} + \eta p_{s,i}^{\text{text}}.$$

This family was attractive because hidden item text is visible at inference time. The hosted scores, mostly around -0.65 to -0.67, showed that text-derived ranking signal was not enough. The failure mode was calibration: a text model might correctly rank two familiar public items but still assign the wrong absolute probability to a new hidden benchmark family.

4.5 Adaptive Labeling And Revealed-Label Adjustment

The competition can reveal a small number of hidden labels before prediction. The adaptive-labeling variants assigned each candidate pair an acquisition score $r_{s,i}$. Two common policies were uncertainty sampling,

$$r_{s,i} = -\left|p_{s,i} - 0.5\right|,$$

and domain coverage sampling, which favored items from underrepresented predicted domains. After labels were revealed, the predictor estimated a small residual correction:

$$\Delta_L = \rho\left(\bar{y}_L - \bar{p}_L\right), \qquad p^L_{s,i} = \mathrm{clip}(p_{s,i} + \Delta_L, \epsilon, 1-\epsilon),$$

where L is the revealed set, $\bar{y}_L$ is the revealed label mean, $\bar{p}_L$ is the model's mean prediction on those revealed rows, and ρ is a shrinkage factor. The hosted evidence did not support aggressive adaptive labeling. The final selected candidate omitted an active acquisition policy and relied on the platform's random revealed labels.

4.6 Item Memory And Exact Lookup

The item-memory family treated exact public item text as an identifier. For a normalized item text key $k(i)$, the method estimated:

$$\hat{p}_i = \frac{n_i \bar{y}_i + \tau_i p^{\text{fallback}}_{s,i}}{n_i + \tau_i},$$

where p^{fallback} was a subject/domain prior used when the item was unseen. In row-holdout validation, this looked strong because many validation rows shared item text with training rows. The audit showed item coverage of 0.8128 in row holdout, but 0.0000 under benchmark-family holdout. Since the hidden competition items come from absent benchmarks, exact public item memory had little reason to transfer. Hosted item-memory variants stayed around -0.63 to -0.64.

4.7 AUC Stacks And Pairwise Rankers

The AUC-focused variants combined several component predictors to maximize ranking quality. A typical stack used a logit opinion pool:

$$p_{s,i} = \sigma\left(\alpha + \gamma \sum_m w_m \ \mathrm{logit}(p^m_{s,i})\right), \qquad w_m \geq 0.$$

Here p^m are component predictions from priors, semantic models, memory models, tree models, or forecast models. The scale γ controls sharpness. Larger γ often improves ranking metrics on public slices because it separates examples more strongly, but it also increases log-loss risk when the hidden base rate shifts.

Pairwise rankers used comparisons rather than direct probabilities. They estimated a latent score $u_{s,i}$ such that

$$\Pr(y_a > y_b) = \sigma(u_a - u_b),$$

then converted ranks back to probabilities. This was aligned with AUC but poorly aligned with negative log-loss. Hosted AUC/rank variants scored around -0.64 to -0.71, with the worst pairwise variant at -0.71.

4.8 Tree Models And Forecast Ensembles

Tree models used structured features from subjects, domains, text cues, public priors, and interaction indicators to predict correctness. Forecast ensembles then combined model outputs through a nonnegative logit pool:

$$z_{s,i} = \alpha + \sum_m w_m \, \mathrm{logit}(p^m_{s,i}), \qquad p_{s,i} = \sigma(z_{s,i}/T),$$

where T is a temperature parameter. On public-proxy validation, these ensembles produced extremely strong local metrics, including log-loss near 0.34 and AUC above 0.928. Hosted results did not transfer, landing around -0.63 to -0.67. The interpretation is that the ensemble learned public benchmark and item structure too well, then became overconfident on shifted hidden benchmark families.

4.9 Compact Domain Priors

The best-transfer family used only broad subject and item-domain signals. Item text was mapped into coarse domain indicators $D_k(i)$, such as coding, math, multimodal, instruction following, safety, or general QA. Each domain effect was shrunk:

$$\delta_k = \frac{n_k}{n_k + \tau_d} \left(\bar{y}_k - c\right), \qquad d(i) = \sum_k D_k(i)\delta_k.$$

The subject term was also shrunk:

$$a_s = \frac{n_s}{n_s + \tau_s} \left(\bar{y}_s - c\right).$$

The compact prediction was:

$$p_{s,i} = \mathrm{clip}\left(c + w_a a_s + w_d d(i), \epsilon, 1 - \epsilon\right).$$

This model avoided exact public item lookup, public benchmark lookup, and high-variance interactions. It was less expressive than the stacks, but it transferred better because it made fewer claims about the unseen benchmark identities. Multiple compact domain-prior variants tied at the best rounded hosted score of -0.62.

4.10 Hierarchical OOD Model

The hierarchical OOD family extended the compact prior with subject-domain interactions and stronger out-of-domain shrinkage:

$$p_{s,i} = \mathrm{clip}\left(c + a_s + d(i) + h_{s,d(i)} + \ell_L, \epsilon, 1 - \epsilon\right).$$

The interaction term $h_{s,d}$ measured whether a subject was unusually strong or weak in a domain after accounting for the subject and domain main effects:

$$h_{s,d} = \frac{n_{s,d}}{n_{s,d} + \tau_h} \left(\bar{y}_{s,d} - c - a_s - d\right).$$

The term ℓ_L represented a conservative revealed-label correction when labels were available. Local cold benchmark-family validation suggested this was a better OOD model than the compact prior. Hosted results did not confirm that improvement: hierarchical OOD submissions ranged from -0.63 to -0.68. The likely reason is that even benchmark-holdout validation did not perfectly match the hosted hidden sample, and the added interaction terms introduced enough variance to hurt log-loss.

4.11 Final Formula Micro-Sweeps

After the hierarchical and high-AUC tracks degraded, the final search returned to the compact formula and changed only calibration-center and strength parameters:

$$p_{s,i}(c, s) = \text{clip}\left(c + s(w_a a_s + w_d d(i)), \epsilon, 1 - \epsilon\right).$$

The sweep tested centers above and below the selected center, including 0.610 and 0.630 above, then 0.590, 0.580, 0.570, 0.560, and 0.550 below. None beat the rounded -0.62 plateau. The lower-center attempts were especially useful because they tested whether hidden labels had a lower base rate than the public evidence suggested. They did not improve hosted log-loss, so the final selection remained the 0.600-centered compact domain prior.

5 Hosted Results

Table 1: Final selected hosted submission.

Field	Value
ZIP	`submission_domain_live_mid600.zip`
Hosted submitted time	2026-06-03 00:26
Hosted negative log-loss	-0.62
Approximate log-loss	0.62
AUC-ROC	0.72 (competition reported)
Runtime family	Compact domain prior: global center, subject alias prior, and coarse item-domain prior

Table 2: Hosted negative log-loss by method family. Scores are leaderboard values; higher is better.

Family	Rows	Best score	Median score	Best ZIP
Compact domain prior	17	-0.62	-0.63	`submission_domain_prior_hidden_cent.zip`
Final domain micro-sweep	8	-0.62	-0.64	`submission_domain_live_mid610.zip`
Alias/scaled priors	15	-0.63	-0.63	`submission_alias_light.zip`
External/domain prior	4	-0.63	-0.63	`submission_external_domain_mid600.zip`
Forecast ensemble	5	-0.63	-0.63	`submission_forecast_robust_logloss_.zip`
Hierarchical OOD	5	-0.63	-0.66	`submission_hier_ood_center600.zip`
Item memory	2	-0.63	-0.64	`submission_item_memory_rank.zip`
AUC/rank stack	6	-0.64	-0.65	`submission_auc_stack_item_push_labe.zip`
Baseline/calibration	7	-0.64	-0.66	`submission_calibrated_constant.zip`
Other	3	-0.64	-0.65	`submission_subject_balanced.zip`
Tree blend	3	-0.64	-0.64	`submission_tree_blend_maxauc_labeli.zip`
OOD gated	2	-0.65	-0.66	`submission_ood_gated.zip`
Semantic/text	3	-0.65	-0.65	`submission_semantic_labeling.zip`

Table 3: All finished submissions tied for the best rounded hosted score in `eval_finals.txt`.

ID	Date	Score	Family	ZIP
777007	2026-06-02 22:26	-0.62	Compact domain prior	`submission_domain_prior_hidden_cent.zip`
777016	2026-06-02 22:28	-0.62	Compact domain prior	`submission_domain_prior_publiccal.zip`
777017	2026-06-02 22:28	-0.62	Compact domain prior	`submission_domain_prior_sweep_loglo.zip`
777204	2026-06-03 00:25	-0.62	Compact domain prior	`submission_domain_live_sweep_s085.zip`
777209	2026-06-03 00:26	-0.62	Compact domain prior	`submission_domain_live_mid600.zip`
777210	2026-06-03 00:26	-0.62	Compact domain prior	`submission_domain_live_mid620.zip`
777211	2026-06-03 00:26	-0.62	Compact domain prior	`submission_domain_live_mid640.zip`
777214	2026-06-03 00:27	-0.62	Compact domain prior	`submission_domain_live_pub_s115.zip`
781993	2026-06-05 15:45	-0.62	Final domain micro-sweep	`submission_domain_live_mid610.zip`

Figure 1 plots all finished hosted scores by submission order. Figure 2 summarizes the best score by method family. Figure 3 shows the cumulative-best search narrative: compact/domain-prior submissions found the -0.62 plateau early, while later high-complexity tracks mostly failed to raise the hosted best.

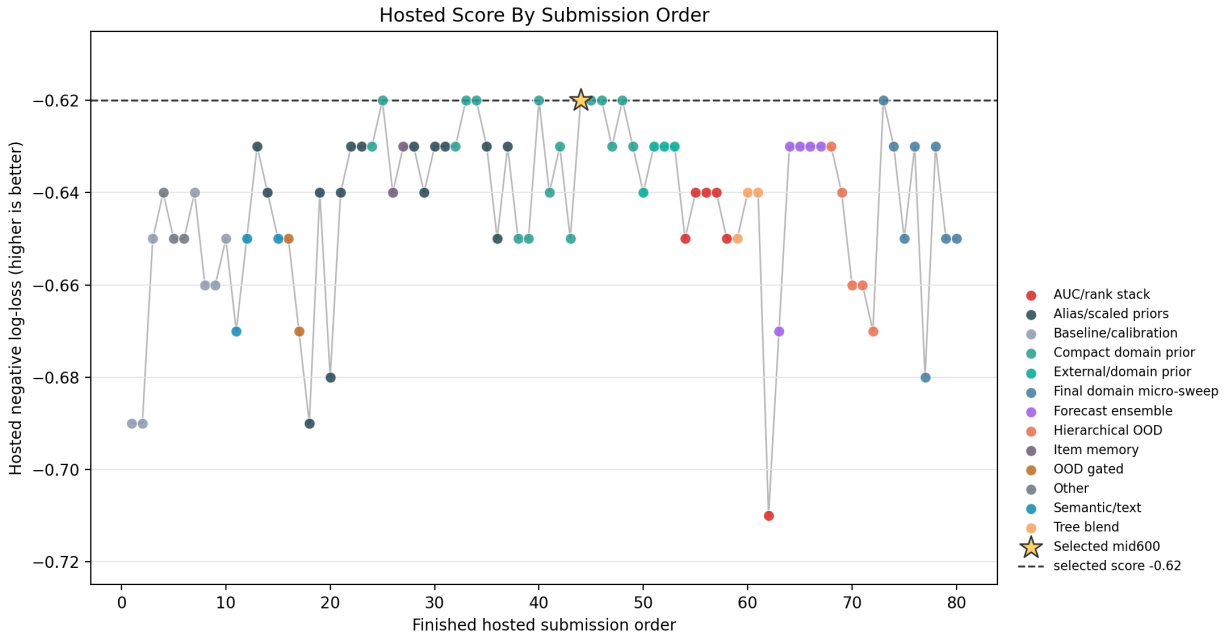

Figure 1: Hosted negative log-loss by finished submission order, colored by method family.

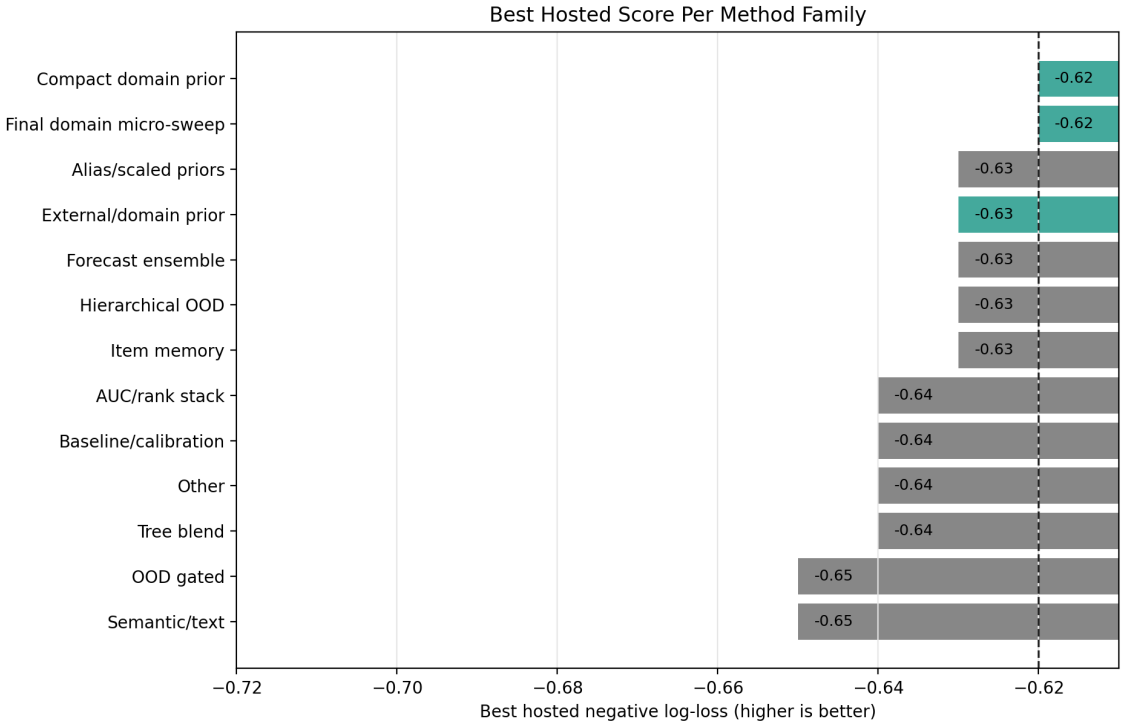

Figure 2: Best rounded hosted score per method family. The dashed line marks the selected score.

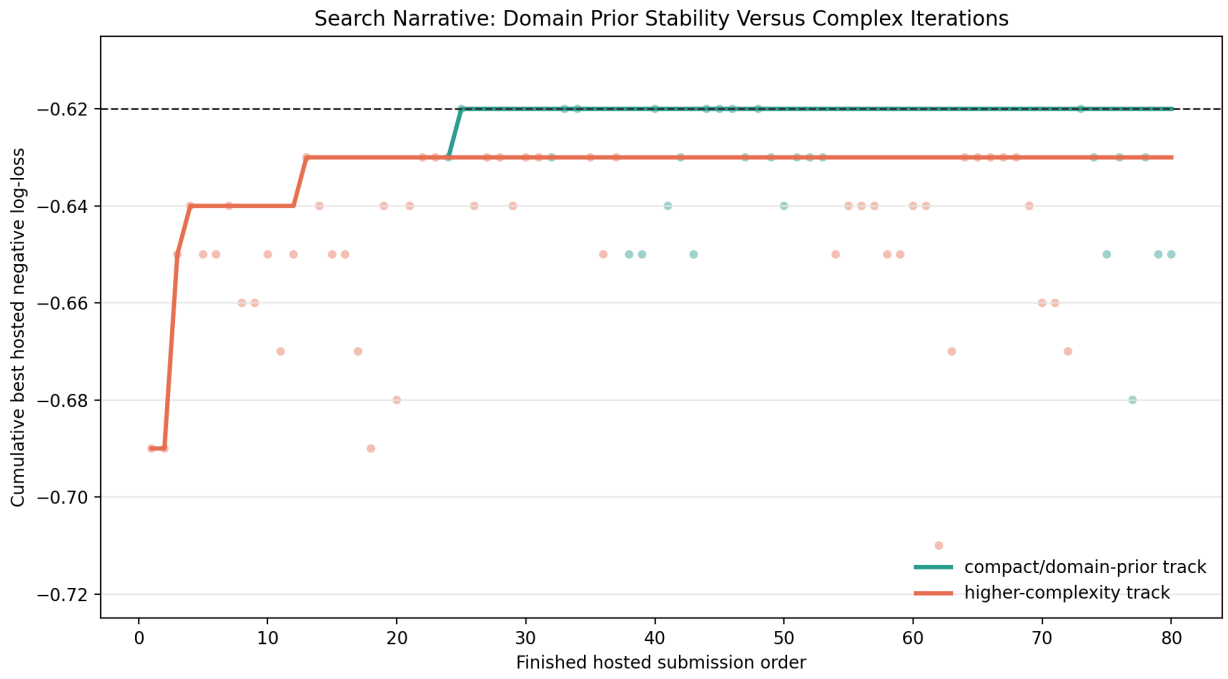

Figure 3: Cumulative best score for compact/domain-prior candidates versus higher-complexity candidates.

6 Why High Local AUC Did Not Transfer

The failed transfer had three interacting causes. Taken together, the hosted evidence shows that public-slice AUC was not reliable promotion evidence for this competition.

First, validation overlap was too permissive. Public-slice probes sampled from the same public matrix used to build many artifacts. This preserved subject, benchmark, condition, and sometimes item overlap. Exact item-memory looked excellent under row holdout because most validation items were still seen, but it had zero item coverage under benchmark holdout.

Second, AUC rewarded ranking but not hidden calibration. AUC-heavy submissions used wide probability spreads. That can improve rank ordering on familiar rows while increasing log-loss when the hidden base rate shifts. The strict benchmark-holdout audit already showed this pattern: weighted log-loss 0.729388 and weighted AUC 0.537691, with several held-out benchmark families showing large prediction-mean errors.

Third, hidden benchmark families changed the base-rate problem. The hidden benchmark identities were absent from public training. A model could learn that a public benchmark or item cluster was easy or hard, but that signal did not directly identify the hidden benchmark family. The compact domain prior was less ambitious, but its shrinkage made it more robust to the unknown hidden base rate.

7 Final Selected Method

The selected method is `submission_domain_live_mid600.zip`. Its formula can be summarized as:

$$p = \text{clip}\left(c + w_s \cdot a(\text{subject}) + w_d \cdot d(\text{item}), \epsilon, 1 - \epsilon\right),$$

where c is a global center near 0.600, $a(\text{subject})$ is a shrunk subject alias prior, and $d(\text{item})$ is a coarse domain-prior adjustment derived from item text. The submitted method carries only the small domain-prior table needed for this formula and does not use an active acquisition policy.

This method did not achieve the competition target of log-loss below 0.60. It was selected because it was the best hosted-risk tradeoff: it tied the best rounded hosted score, had competition-reported AUC-ROC 0.72, and remained stable across neighboring variants while more complex local winners degraded.

8 Lessons Learned And Future Work

The primary lesson is that hosted negative log-loss must be treated as the objective, not public-proxy AUC. Future work should use benchmark-family holdout as the default promotion gate and should reject candidates that improve ranking by increasing probability spread unless they also improve cold calibration.

The most promising future direction is not another exact-memory or tree-stack attempt. It is a better hidden base-rate estimator: richer but strongly shrunk item-domain signatures, subject-domain interactions validated only under leave-benchmark-family-out folds, and calibration centers selected for worst-fold robustness rather than public average performance. Adaptive labeling should remain disabled unless a policy consistently beats platform-random labels under benchmark-holdout simulation and does not worsen the worst fold.

9 Clean Publication Repository

The clean publication artifact is organized as `aims-competition` and published at https://github.com/BlakeMasters/aims-competition. It contains the final uploadable ZIP, an extracted copy of the final submission source for inspection, archived alternate submission ZIPs under `unused_submissions`, this technical report as both PDF and LaTeX source, generated report figures and tables, hosted-results files, minimal starter-kit validation tools, and the tiny sample files required for local smoke testing. It intentionally excludes validation extraction folders, Python caches, raw downloaded benchmark data, virtual environments, agent instructions, documentation folders, and iterative working artifacts.

This separation matters because the final competition evidence should be easy to inspect without requiring an evaluator to navigate the full experimental workspace. The clean repository is therefore the publication unit, while the larger working directory remains an audit trail of the search process.

10 Reproducibility

The hosted-results tables and figures were regenerated from `eval_finals.txt` and local analysis notes. The parser found 82 hosted rows and 80 finished scored rows. The best rounded hosted score was -0.62, tied by 9 finished rows in the cleaned hosted result table. The derived CSV files under `report/tables/` provide the exact counts used in Table 2.

References

[1] Aims foundations measurement db dataset. https://huggingface.co/datasets/aims-foundations/measurement-db, 2026. Public training dataset referenced by the starter kit notes.

[2] Predictive ai evaluation challenge starter kit notes. Local competition starter kit file: `asn.txt`, 2026. Defines the runtime interface, hidden-evaluation setup, and primary negative log-loss metric.

[3] Predictive evaluation competition. https://www.codabench.org/competitions/15934/, 2026. Codabench competition page for the hosted Predictive Evaluation Competition.